

Microservices Architecture Documentation

1. Introduction

Dokumentasi ini dibuat untuk menjelaskan implementasi arsitektur microservices pada aplikasi Kelarin. Pada modul ini, backend monolith dipecah menjadi beberapa service independen agar sistem lebih modular, scalable, dan mudah dikelola.

Arsitektur microservices pada project ini terdiri dari:

- Auth Service
- Item Service
- API Gateway (Nginx)
- Database terpisah untuk setiap service
- Frontend Application

2. Architecture Overview

Microservices Architecture Diagram

```
graph TD
    Frontend --> Gateway
    Gateway --> AuthService
    Gateway --> ItemService
    AuthService --> AuthDB
    ItemService --> ItemDB
```

3. Service Overview

Service	Port	Description
Frontend	80	User interface aplikasi
Gateway (Nginx)	80	API Gateway dan reverse proxy
Auth Service	8001	Authentication dan authorization service
Item Service	8002	CRUD item management
Auth Database	5432	Database khusus authentication

Service	Port	Description
Item Database	5432	Database khusus item service

4. API Contract

Auth Service

Method	Endpoint	Description
POST	<code>/register</code>	Register user baru
POST	<code>/login</code>	Login user
GET	<code>/verify</code>	Verifikasi JWT token
GET	<code>/health</code>	Healthcheck auth-service

Item Service

Method	Endpoint	Description
GET	<code>/items</code>	Menampilkan seluruh item
POST	<code>/items</code>	Menambahkan item baru
PUT	<code>/items/{id}</code>	Mengubah item
DELETE	<code>/items/{id}</code>	Menghapus item
GET	<code>/health</code>	Healthcheck item-service

5. Docker Compose Workflow

Command	Description
<code>docker compose up --build</code>	Menjalankan seluruh container microservices
<code>docker compose down</code>	Menghentikan seluruh container
<code>docker compose ps</code>	Melihat status container
<code>docker compose logs auth-service</code>	Melihat logs auth-service
<code>docker compose logs item-service</code>	Melihat logs item-service

6. Healthcheck & Monitoring

Health Endpoint

Service	Endpoint	Status
Auth Service	/health	☒ Active
Item Service	/health	☒ Active

Monitoring Logs

Command	Function
<code>docker compose logs auth-service</code>	Monitoring auth-service
<code>docker compose logs item-service</code>	Monitoring item-service

7. Testing Validation

Docker Compose Validation

Service	Status
auth-service	☒ Healthy
item-service	☒ Healthy
auth-db	☒ Healthy
item-db	☒ Healthy
frontend	☒ Running
gateway-nginx	☒ Running

Keterangan

- Seluruh container berhasil dijalankan menggunakan Docker Compose.
- Healthcheck service menunjukkan status healthy.
- Tidak ditemukan crash pada service utama.

Health Endpoint Testing

Test Case	Result
Auth Service /health accessible	☒ Passed
Item Service /health accessible	☒ Passed
Health status returns 200 OK	☒ Passed

Keterangan

- Endpoint health berhasil diakses melalui Swagger dan Docker healthcheck.
- Service memberikan response **200 OK**.
- Monitoring service berjalan dengan normal tanpa error.

Feature Testing

Feature	Result
Register User	☑ Passed
Login User	☑ Passed
Create Item	☑ Passed
Read Item	☑ Passed
Update Item	☑ Passed
Delete Item	☑ Passed

Keterangan

- Seluruh fitur utama aplikasi berhasil berjalan pada implementasi microservices.
- Frontend berhasil terhubung dengan API Gateway dan backend services.

8. Development vs Microservices Comparison

Component	Monolith Architecture	Microservices Architecture
Backend Structure	Single backend	Multiple independent services
Database	Single database	Database per service
Deployment	Single container	Multi-container
Scalability	Limited	More scalable
Fault Isolation	Single point of failure	Service isolation

9. Documentation Evidence

Testing Activity	Evidence	Description
------------------	----------	-------------

Testing Activity

Evidence

Description

Docker
Compose
Validation

```
(base) PS C:\Users\DELL 4510\cc-kelompok-a-ethereal> docker compose ps
NAME                COMMAND                                SERVICE    CREATED      STATUS      PORTS
kelompok-kelarin-backend-v2  "kelompok-kelarin-backend-v2"  backend    2 hours ago  Up 2 hours  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
db-kelarin           "postgres-16-alpine"            db         2 hours ago  Up 2 hours  0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
frontend             "kelompok-kelarin-frontend-v1"  frontend   2 hours ago  Up 2 hours  0.0.0.0:5177->5177/tcp, [::]:5177->5177/tcp
```

Pengujian dilakukan menggunakan perintah **docker compose ps** untuk memastikan seluruh container microservices berjalan dengan status running dan healthy.

Auth Service
Healthcheck

```
(base) PS C:\Users\DELL 4510\cc-kelompok-a-ethereal> docker compose logs auth-service
auth-service | INFO: 127.0.0.1:37874 - "GET /health HTTP/1.1" 200 OK
auth-service | INFO: 127.0.0.1:44890 - "GET /health HTTP/1.1" 200 OK
auth-service | INFO: 127.0.0.1:43076 - "GET /health HTTP/1.1" 200 OK
auth-service | INFO: 127.0.0.1:53478 - "GET /health HTTP/1.1" 200 OK
auth-service | INFO: 127.0.0.1:40536 - "GET /health HTTP/1.1" 200 OK
auth-service | INFO: 127.0.0.1:53346 - "GET /health HTTP/1.1" 200 OK
auth-service | INFO: 127.0.0.1:37816 - "GET /health HTTP/1.1" 200 OK
auth-service | INFO: 127.0.0.1:52392 - "GET /health HTTP/1.1" 200 OK
auth-service | INFO: 127.0.0.1:33108 - "GET /health HTTP/1.1" 200 OK
auth-service | INFO: 127.0.0.1:33842 - "GET /health HTTP/1.1" 200 OK
auth-service | INFO: 127.0.0.1:39268 - "GET /health HTTP/1.1" 200 OK
auth-service | INFO: 127.0.0.1:44996 - "GET /health HTTP/1.1" 200 OK
auth-service | INFO: 127.0.0.1:41498 - "GET /health HTTP/1.1" 200 OK
auth-service | INFO: 127.0.0.1:58068 - "GET /health HTTP/1.1" 200 OK
auth-service | INFO: 127.0.0.1:32836 - "GET /health HTTP/1.1" 200 OK
auth-service | INFO: 127.0.0.1:49588 - "GET /health HTTP/1.1" 200 OK
```

Pengujian dilakukan pada endpoint **/health** auth-service melalui Swagger untuk memastikan service authentication berjalan normal dengan response **200 OK**.

Item Service
Healthcheck

```
(base) PS C:\Users\DELL 4510\cc-kelompok-a-ethereal> docker compose logs item-service
item-service | INFO: 127.0.0.1:53704 - "GET /health HTTP/1.1" 200 OK
item-service | INFO: 127.0.0.1:58378 - "GET /health HTTP/1.1" 200 OK
item-service | INFO: 127.0.0.1:37868 - "GET /health HTTP/1.1" 200 OK
item-service | INFO: 127.0.0.1:50884 - "GET /health HTTP/1.1" 200 OK
item-service | INFO: 127.0.0.1:48380 - "GET /health HTTP/1.1" 200 OK
item-service | INFO: 127.0.0.1:52548 - "GET /health HTTP/1.1" 200 OK
item-service | INFO: 127.0.0.1:35114 - "GET /health HTTP/1.1" 200 OK
item-service | INFO: 127.0.0.1:53528 - "GET /health HTTP/1.1" 200 OK
item-service | INFO: 127.0.0.1:32978 - "GET /health HTTP/1.1" 200 OK
item-service | INFO: 127.0.0.1:39716 - "GET /health HTTP/1.1" 200 OK
item-service | INFO: 127.0.0.1:58206 - "GET /health HTTP/1.1" 200 OK
item-service | INFO: 127.0.0.1:50874 - "GET /health HTTP/1.1" 200 OK
item-service | INFO: 127.0.0.1:44788 - "GET /health HTTP/1.1" 200 OK
item-service | INFO: 127.0.0.1:34696 - "GET /health HTTP/1.1" 200 OK
item-service | INFO: 127.0.0.1:38276 - "GET /health HTTP/1.1" 200 OK
item-service | INFO: 127.0.0.1:53812 - "GET /health HTTP/1.1" 200 OK
```

Pengujian dilakukan pada endpoint **/health** item-service untuk memastikan service item management berjalan normal tanpa error.

User
Registration
Testing

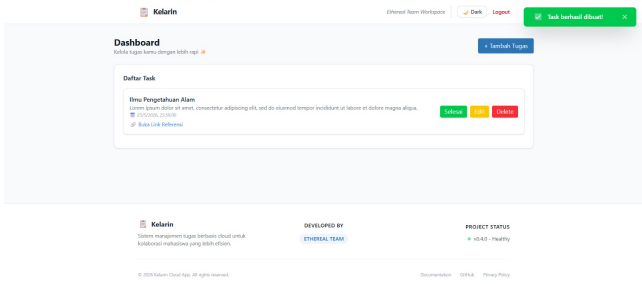
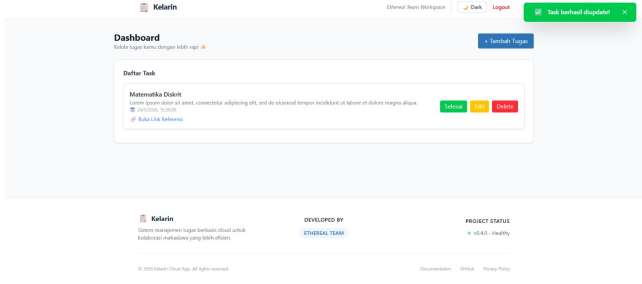
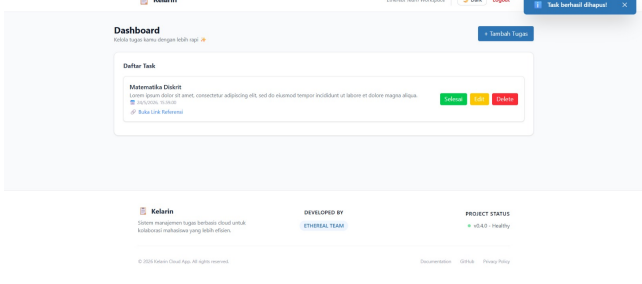
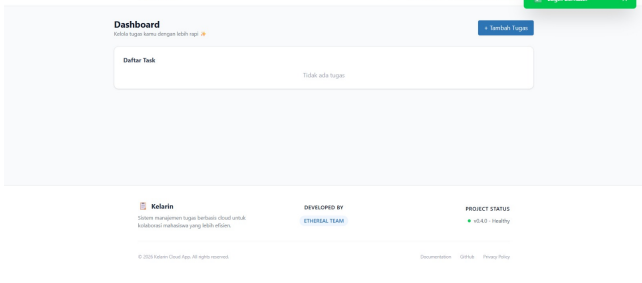
Pengujian dimulai dengan membuat akun baru melalui halaman registrasi untuk memastikan auth-service dapat menyimpan data user dengan baik.

User Login
Testing

Pengujian dilakukan menggunakan akun yang telah terdaftar untuk memastikan proses authentication dan JWT token berjalan normal.

Create Item
Testing

Pengujian dilakukan dengan menambahkan item baru pada aplikasi untuk memastikan item-service dapat menerima dan menyimpan data dengan baik.

Testing Activity	Evidence	Description
Read Item Testing		Pengujian dilakukan untuk memastikan item yang telah dibuat berhasil ditampilkan pada frontend application.
Update Item Testing		Pengujian dilakukan dengan mengubah data item untuk memastikan fitur update berjalan normal pada microservices architecture.
Delete Item Testing		Pengujian dilakukan dengan menghapus item dari sistem untuk memastikan delete endpoint berjalan dengan baik tanpa error.
Frontend Microservices Testing		Pengujian dilakukan dengan membuka aplikasi melalui gateway nginx untuk memastikan frontend berhasil terhubung dengan backend microservices.

10. Debugging Guide

Problem	Solution
Container tidak berjalan	Jalankan <code>docker compose up --build</code>
Service unhealthy	Cek logs menggunakan <code>docker compose logs <service-name></code>
Frontend tidak dapat connect backend	Periksa konfigurasi gateway nginx
Endpoint tidak dapat diakses	Pastikan container running dan network Docker aktif

11. Conclusion

Implementasi microservices pada aplikasi Kelarin berhasil dijalankan menggunakan Docker Compose dengan arsitektur multi-service.

Seluruh service:

- berhasil berjalan
- saling terhubung
- memiliki healthcheck
- dapat melakukan komunikasi antar service dengan normal

Pengujian fitur utama aplikasi menunjukkan bahwa sistem berjalan stabil pada implementasi microservices.

Final Status

● MICROSERVICES ARCHITECTURE RUNNING SUCCESSFULLY